

June 27, 2026

1 Phân tích Du lịch Quốc tế Việt Nam

1.1 Vietnam International Tourism Arrivals Analysis (2009–2026)

Data source: Quarterly arrivals by country (Q1–Q4), extracted from HTML-Excel files.

Note: 2021 data is absent (COVID gap). Q3/Q4 files include 2008; Q1/Q2 start from 2009.

```
[ ]: import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from lxml import html
import warnings
warnings.filterwarnings("ignore")
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["figure.dpi"] = 100
sns.set_style("whitegrid")
print("All imports successful.")
```

1.2 1. Data Parsing

```
[ ]: def parse_quarterly_file(filepath, quarter_label):
    with open(filepath, "r", encoding="utf-8") as f:
        raw = f.read()
        tree = html.fromstring(raw)
        rows = tree.xpath("//tr")
        header_tds = rows[0].xpath("./td")
        years = [(td.tail or "").strip() for td in header_tds[2:]]
        records = []
        for row in rows[1:]:
            tds = row.xpath("./td")
            if len(tds) < 3: continue
            country = (tds[0].tail or "").strip()
            if not country or country == "Phân theo thị trường": continue
            for j, td in enumerate(tds[1:]):
                val_str = (td.tail or "").strip()
```

```

        year_label = years[j] if j < len(years) else None
        if not val_str or not year_label or year_label == "Totals": continue
        try: val = float(val_str.replace(".", ""))
        except ValueError: continue
        records.append({"country": country, "year": int(year_label),
↪ "quarter": quarter_label, "arrivals": val})
    return pd.DataFrame(records)

files = [("Q1", "quy1-cacnuoc.xls"), ("Q2", "quy2-cacnuoc.xls"), ("Q3",
↪ "quy3-cacnuoc.xls"), ("Q4", "quy4-cacnuoc.xls")]
dfs = []
for label, fpath in files:
    df = parse_quarterly_file(fpath, label)
    dfs.append(df)
    print(f"{label}: {len(df)} records, {df['country'].nunique()} countries")
df_long = pd.concat(dfs, ignore_index=True)
hk = df_long[(df_long["country"] == "Hoa Kỳ") & (df_long["year"] == 2009) &
↪ (df_long["quarter"] == "Q1")]
assert hk["arrivals"].values[0] == 104520.0
print(f"Verification: Hoa Kỳ 2009 Q1 = {hk['arrivals'].values[0]:.0f}")

```

1.3 2. Data Cleaning

```

[ ]: df_long = df_long[df_long["country"] != "Totals"].reset_index(drop=True)
df_total = df_long.groupby(["country", "year"])["arrivals"].sum().reset_index()
df_total.columns = ["country", "year", "total_arrivals"]
df_long["arrivals"] = df_long["arrivals"].fillna(0)
df_total["total_arrivals"] = df_total["total_arrivals"].fillna(0)
all_countries = sorted(df_long["country"].unique())
all_years = sorted(df_long["year"].unique())
idx = pd.MultiIndex.from_product([all_countries, all_years,
↪ ["Q1", "Q2", "Q3", "Q4"]], names=["country", "year", "quarter"])
df_complete = df_long.set_index(["country", "year", "quarter"]).reindex(idx,
↪ fill_value=0).reset_index()
print(f"df_complete: {df_complete.shape}, {len(all_countries)} countries,
↪ {len(all_years)} years")

```

1.4 3. Feature Engineering & Train-Test Split

```

[ ]: quarter_map = {"Q1": 1, "Q2": 2, "Q3": 3, "Q4": 4}
df_complete["quarter_num"] = df_complete["quarter"].map(quarter_map)
df_complete["time_idx"] = df_complete["year"] + (df_complete["quarter_num"] -
↪ 1) / 4
df_complete = df_complete.sort_values(["country", "year", "quarter_num"])
for lag in [1, 2, 4]:

```

```

df_complete[f"lag_{lag}"] = df_complete.groupby("country")["arrivals"].
↳shift(lag)
df_complete["rolling_mean_4"] = df_complete.groupby("country")["arrivals"].
↳transform(lambda x: x.rolling(4, min_periods=1).mean())
TRAIN_END = 2023
TEST_START = 2024
df_train = df_complete[df_complete["year"] <= TRAIN_END].copy()
df_test = df_complete[df_complete["year"] >= TEST_START].copy()
print(f"Train: {df_train.shape[0]} rows, Test: {df_test.shape[0]} rows")

```

1.5 4. Exploratory Data Analysis

```

[ ]: fig, ax = plt.subplots(figsize=(14, 7))
yearly_total = df_total.groupby("year")["total_arrivals"].sum().reset_index()
ax.plot(yearly_total["year"], yearly_total["total_arrivals"] / 1e6, "o-", lw=2,
↳ms=8, color="#2196F3")
ax.set_xlabel("Year"); ax.set_ylabel("Total Arrivals (millions)")
ax.set_title("Vietnam International Tourist Arrivals by Year",
↳fontweight="bold")
ax.set_xticks(yearly_total["year"]); ax.set_xticklabels(yearly_total["year"],
↳rotation=45)
plt.tight_layout(); plt.savefig("eda_total_trend.png", dpi=150,
↳bbox_inches="tight"); plt.show()

```

```

[ ]: top10 = df_total.groupby("country")["total_arrivals"].sum().
↳sort_values(ascending=False).head(10)
fig, ax = plt.subplots(figsize=(12, 7))
bars = ax.barh(range(len(top10)), top10.values / 1e6, color=sns.
↳color_palette("viridis", len(top10)))
ax.set_yticks(range(len(top10))); ax.set_yticklabels(top10.index, fontsize=11)
ax.set_title("Top 10 Source Countries", fontweight="bold"); ax.invert_yaxis()
plt.tight_layout(); plt.savefig("eda_top10_countries.png", dpi=150,
↳bbox_inches="tight"); plt.show()

```

```

[ ]: fig, ax = plt.subplots(figsize=(10, 6))
quarterly = df_long.groupby("quarter")["arrivals"].agg(["mean", "std"]).
↳reindex(["Q1", "Q2", "Q3", "Q4"])
ax.bar(quarterly.index, quarterly["mean"]/1e3, yerr=quarterly["std"]/1e3,
↳capsize=5,
color=["#FF6B6B", "#4ECDC4", "#45B7D1", "#96CEB4"], edgecolor="black")
ax.set_title("Seasonality: Average Arrivals by Quarter", fontweight="bold")
plt.tight_layout(); plt.savefig("eda_seasonality.png", dpi=150,
↳bbox_inches="tight"); plt.show()

```

```

[ ]: top5 = top10.head(5).index.tolist()

```

```

pivot = df_total[df_total["country"].isin(top5)].pivot_table(index="year",
    ↪columns="country", values="total_arrivals")
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(pivot.corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0,
    ↪ax=ax, square=True)
ax.set_title("Correlation Between Top 5 Countries", fontweight="bold")
plt.tight_layout(); plt.savefig("eda_correlation.png", dpi=150,
    ↪bbox_inches="tight"); plt.show()

```

```

[ ]: fig, axes = plt.subplots(3, 2, figsize=(16, 14))
for i, c in enumerate(top5):
    d = df_total[df_total["country"] == c].sort_values("year")
    axes.flatten()[i].plot(d["year"], d["total_arrivals"]/1e3, "o-", lw=2)
    axes.flatten()[i].set_title(c, fontweight="bold")
axes.flatten()[5].set_visible(False)
plt.suptitle("Country-Specific Trends", fontweight="bold", y=1.01)
plt.tight_layout(); plt.savefig("eda_country_trends.png", dpi=150,
    ↪bbox_inches="tight"); plt.show()

```

1.6 5. Modeling

```

[ ]: from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX

agg_all = df_complete.groupby(["year", "quarter_num", "time_idx"])["arrivals"].
    ↪sum().reset_index().sort_values(["year", "quarter_num"])
agg_all["lag_1"] = agg_all["arrivals"].shift(1)
agg_all["lag_4"] = agg_all["arrivals"].shift(4)
agg_all["rolling_mean_4"] = agg_all["arrivals"].rolling(4, min_periods=1).mean()
feat = ["year", "quarter_num", "time_idx", "lag_1", "lag_4", "rolling_mean_4"]
tr = agg_all[agg_all["year"] <= TRAIN_END].dropna(subset=["lag_1", "lag_4"])
te = agg_all[agg_all["year"] >= TEST_START].dropna(subset=["lag_1", "lag_4"])
X_train, y_train = tr[feat].values, tr["arrivals"].values
X_test, y_test = te[feat].values, te["arrivals"].values

lr = LinearRegression().fit(X_train, y_train)
y_lr = lr.predict(X_test); mae_lr = mean_absolute_error(y_test, y_lr); rmse_lr
    ↪= np.sqrt(mean_squared_error(y_test, y_lr)); r2_lr = r2_score(y_test, y_lr)
rf = RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42).
    ↪fit(X_train, y_train)
y_rf = rf.predict(X_test); mae_rf = mean_absolute_error(y_test, y_rf); rmse_rf
    ↪= np.sqrt(mean_squared_error(y_test, y_rf)); r2_rf = r2_score(y_test, y_rf)
xgb_m = xgb.XGBRegressor(n_estimators=200, max_depth=6, learning_rate=0.1,
    ↪random_state=42).fit(X_train, y_train)

```

```

y_xgb = xgb_m.predict(X_test); mae_xgb = mean_absolute_error(y_test, y_xgb);
    ↳rmse_xgb = np.sqrt(mean_squared_error(y_test, y_xgb)); r2_xgb =
    ↳r2_score(y_test, y_xgb)

agg_ts = df_complete[df_complete["year"] <= TRAIN_END].
    ↳groupby(["year", "quarter_num"])["arrivals"].sum().reset_index().
    ↳sort_values(["year", "quarter_num"])
agg_ts = agg_ts[agg_ts["year"] != 2021]
sarima = SARIMAX(agg_ts["arrivals"].values, order=(1,1,1),
    ↳seasonal_order=(1,1,1,4), enforce_stationarity=False,
    ↳enforce_invertibility=False).fit(dispatch=False, maxiter=500)
test_ts = df_complete[df_complete["year"] >= TEST_START].
    ↳groupby(["year", "quarter_num"])["arrivals"].sum().reset_index().
    ↳sort_values(["year", "quarter_num"])
sp = sarima.forecast(steps=len(test_ts))
y_s = test_ts["arrivals"].values[:len(sp)]
mae_s = mean_absolute_error(y_s, sp); rmse_s = np.sqrt(mean_squared_error(y_s,
    ↳sp)); r2_s = r2_score(y_s, sp)

comparison = pd.DataFrame({"Model": ["Linear Regression", "Random
    ↳Forest", "XGBoost", "SARIMA"], "MAE": [mae_lr, mae_rf, mae_xgb, mae_s], "RMSE":
    ↳[rmse_lr, rmse_rf, rmse_xgb, rmse_s], "R²": [r2_lr, r2_rf, r2_xgb, r2_s]})
print(comparison.to_string(index=False))

```

1.7 6. Hyperparameter Optimization

```

[ ]: from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

rf_gs = GridSearchCV(RandomForestRegressor(random_state=42), {"n_estimators":
    ↳[100,200,300], "max_depth": [5,10,15,None], "min_samples_split": [2,5,10]},
    ↳cv=3, scoring="neg_mean_absolute_error", n_jobs=-1).fit(X_train, y_train)
y_rf_gs = rf_gs.predict(X_test)
mae_rf_gs = mean_absolute_error(y_test, y_rf_gs); rmse_rf_gs = np.
    ↳sqrt(mean_squared_error(y_test, y_rf_gs)); r2_rf_gs = r2_score(y_test,
    ↳y_rf_gs)
print(f"Best RF: {rf_gs.best_params}")

xgb_rs = RandomizedSearchCV(xgb.XGBRegressor(random_state=42), {"n_estimators":
    ↳[100,200,300,500], "max_depth": [3,5,7,9], "learning_rate": [0.01,0.05,0.1,0.
    ↳2], "subsample": [0.7,0.8,0.9,1.0], "colsample_bytree": [0.7,0.8,0.9,1.0]},
    ↳n_iter=50, cv=3, scoring="neg_mean_absolute_error", n_jobs=-1,
    ↳random_state=42).fit(X_train, y_train)
y_xgb_rs = xgb_rs.predict(X_test)
mae_xgb_rs = mean_absolute_error(y_test, y_xgb_rs); rmse_xgb_rs = np.
    ↳sqrt(mean_squared_error(y_test, y_xgb_rs)); r2_xgb_rs = r2_score(y_test,
    ↳y_xgb_rs)

```

```

print(f"Best XGB: {xgb_rs.best_params}")

comparison_v2 = pd.DataFrame({"Model": ["Linear Regression", "Random
↳Forest", "XGBoost", "SARIMA", "RF (optimized)", "XGBoost (optimized)"], "MAE":
↳[mae_lr, mae_rf, mae_xgb, mae_s, mae_rf_gs, mae_xgb_rs], "RMSE":
↳[rmse_lr, rmse_rf, rmse_xgb, rmse_s, rmse_rf_gs, rmse_xgb_rs], "R²":
↳[r2_lr, r2_rf, r2_xgb, r2_s, r2_rf_gs, r2_xgb_rs]})
print(comparison_v2.to_string(index=False))

```

1.8 7. Forecasting Next 4 Quarters

```

[ ]: full_ts = df_complete.groupby(["year", "quarter_num"])["arrivals"].sum().
↳reset_index().sort_values(["year", "quarter_num"])
full_ts = full_ts[full_ts["year"] != 2021]
s_full = SARIMAX(full_ts["arrivals"].values, order=(1,1,1),
↳seasonal_order=(1,1,1,4), enforce_stationarity=False,
↳enforce_invertibility=False).fit(dispatch=False, maxiter=500)
fc = s_full.get_forecast(steps=4)
fc_mean = np.array(fc.predicted_mean).flatten()
fc_ci = np.array(fc.conf_int(alpha=0.05))
ly = full_ts["year"].max(); lq = full_ts[full_ts["year"]==ly]["quarter_num"].
↳max()
fqs = []; y, q = ly, lq
for _ in range(4):
    q += 1
    if q > 4: q = 1; y += 1
    fqs.append((y, q))
for (yr, qr), v, lo, hi in zip(fqs, fc_mean, fc_ci[:,0], fc_ci[:,1]):
    print(f" {yr} Q{qr}: {v:,.0f} [{lo:,.0f} - {hi:,.0f}]")

fig, ax = plt.subplots(figsize=(14, 7))
hist = full_ts.copy(); hist["lbl"] = hist["year"].astype(str) + "Q" +
↳hist["quarter_num"].astype(str)
ax.plot(range(len(hist)), hist["arrivals"]/1e6, "o-", lw=2, ms=5,
↳color="#2196F3", label="Historical")
fcx = range(len(hist), len(hist)+4)
ax.plot(fcx, fc_mean/1e6, "s-", lw=2, ms=8, color="#FF5722", label="Forecast")
ax.fill_between(fcx, fc_ci[:,0]/1e6, fc_ci[:,1]/1e6, alpha=0.3,
↳color="#FF5722", label="95% CI")
ax.axvline(x=len(hist)-0.5, color="gray", ls="--", alpha=0.5)
ax.set_title("Historical & SARIMA Forecast", fontweight="bold"); ax.legend();
↳ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig("forecast_plot.png", dpi=150,
↳bbox_inches="tight"); plt.show()

```

1.9 8. Save Outputs

```
[ ]: df_long.to_csv("df_long.csv", index=False)
df_total.to_csv("df_total.csv", index=False)
comparison_v2.to_csv("model_comparison.csv", index=False)
pd.DataFrame({"year": [y for y,q in fqs], "quarter": [f"Q{q}" for y,q in fqs],
    ↪ "forecast": fc_mean, "ci_lower": fc_ci[:,0], "ci_upper": fc_ci[:,1]}).
    ↪to_csv("forecast.csv", index=False)
print("All outputs saved.")
```